WORKSHOP PRACTICO

API Testing con Bruno

Fundamentos, Automatizacion e Integracion con IA para Testers Principiantes

Nivel: Principiante

Duracion: 4.5 horas

Herramienta: Bruno + JSONPlaceholder

2025

Tabla de Contenidos

1. Fundamentos de API Testing

1.1 Que es una API

1.2 Tipos de APIs y Protocolos

1.3 HTTP Methods y Status Codes

1.4 Estructura de Request y Response

2. Casos de Prueba y Generacion con IA

2.1 Diseno de Casos de Prueba

2.2 Prompts Efectivos para IA

2.3 Generacion de Datos Mock

3. Bruno: Guia Completa

3.1 Instalacion y Primeros Pasos

3.2 Workspaces y Collections

3.3 Variables y Environments

3.4 Pre-scripts y Test Scripts

3.5 Data-Driven Testing

3.6 Exportar y Compartir

4. Buenas Practicas y Errores Comunes

1. Fundamentos de API Testing

1.1 Que es una API

Una **API (Application Programming Interface)** es un conjunto de reglas y protocolos que permite que diferentes aplicaciones de software se comuniquen entre si. Esencialmente, una API define como los componentes de software deben interactuar.

> **Analogia del Restaurante**
>
> Imagina un restaurante: tu eres el cliente (aplicacion), la cocina es el servidor (backend), y el mesero es la API. Tu no vas directamente a la cocina; el mesero (API) lleva tu orden y trae tu comida. La API es el intermediario que traduce tu solicitud en algo que el servidor entiende.

Por que es importante el API Testing?

- **Calidad temprana:** Detecta defectos antes de la integracion UI
- **Velocidad:** Las pruebas de API son mas rapidas que las pruebas UI
- **Estabilidad:** Las APIs cambian menos frecuentemente que las interfaces
- **Cobertura:** Permite probar logica de negocio directamente
- **Automatizacion:** Facil de integrar en pipelines CI/CD

1.2 Tipos de APIs y Protocolos

Tabla 1 Comparativa de Tipos de API

Tipo	Protocolo	Formato	Cuando Usar
REST	HTTP/HTTPS	JSON, XML	Aplicaciones web modernas, microservicios
SOAP	HTTP, SMTP	XML estricto	Enterprise, banca, sistemas legacy
GraphQL	HTTP	JSON	Apps con datos complejos y relaciones
gRPC	HTTP/2	Protocol Buffers	Comunicacion entre microservicios

Este workshop se enfoca en **REST APIs con formato JSON**, el estandar mas utilizado en la industria actual.

1.3 HTTP Methods y Status Codes

Metodos HTTP

Tabla 2 Metodos HTTP y su Proposito

Metodo	Accion	Ejemplo	Idempotente
GET	Obtener recurso	GET /users/1	Si
POST	Crear recurso	POST /users	No
PUT	Actualizar completo	PUT /users/1	Si
PATCH	Actualizar parcial	PATCH /users/1	No
DELETE	Eliminar recurso	DELETE /users/1	Si

Status Codes HTTP

Tabla 3 Categorias de Status Codes

Rango	Categoria	Ejemplos Comunes
2xx	Exito	200 OK, 201 Created, 204 No Content
3xx	Redireccion	301 Moved, 304 Not Modified
4xx	Error Cliente	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Error Servidor	500 Internal Error, 502 Bad Gateway, 503 Unavailable

1.4 Estructura de Request y Response

Componentes de un HTTP Request

```
GET /users/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
Authorization: Bearer eyJhbGciOiJ...
Content-Type: application/json
Accept: application/json

// Body (solo para POST/PUT/PATCH)
{
  "name": "Ana Garcia",
  "email": "ana@example.com"
}
```

- **Linea de Request:** Metodo + URL + Version HTTP
- **Headers:** Metadatos de la peticion (autenticacion, formato, etc.)
- **Body:** Datos enviados (JSON, XML, form-data)

Componentes de un HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 256
Cache-Control: max-age=3600

{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "city": "Gwenborough"
  }
}
```

- **Status Line:** Version HTTP + Codigo + Mensaje
- **Headers:** Metadatos de la respuesta
- **Body:** Datos devueltos por el servidor

2. Casos de Prueba y Generacion con IA

2.1 Diseno de Casos de Prueba

Un caso de prueba de API debe incluir:

Tabla 4 Estructura de un Caso de Prueba API

Elemento	Descripcion	Ejemplo
ID	Identificador unico	API_USER_001
Descripcion	Que se esta probando	Obtener usuario existente
Precondiciones	Estado previo necesario	Usuario con ID=1 existe
Request	Datos de entrada	GET /users/1
Resultado Esperado	Validaciones	Status 200, JSON con campos requeridos

Tecnica: Particion de Equivalencia

Divide los datos de entrada en grupos que deben comportarse igual. Por ejemplo, para un campo "edad": valores validos (18-65), valores invalidos (menor 18, mayor 65), y valores limite (17, 18, 65, 66).

2.2 Prompts Efectivos para IA

La IA puede acelerar significativamente la creacion de casos de prueba. El secreto esta en prompts bien estructurados:

Prompt Template para Casos de Prueba

```
Actua como un QA Engineer senior especializado en API Testing.

Contexto:
- API REST para gestion de usuarios
- Base URL: https://api.example.com
- Autenticacion: Bearer token

Endpoints a probar:
1. GET /users - Listar usuarios
2. GET /users/{id} - Obtener usuario por ID
3. POST /users - Crear usuario
4. PUT /users/{id} - Actualizar usuario
5. DELETE /users/{id} - Eliminar usuario

Genera 10 casos de prueba que incluyan:
- Casos positivos (happy path)
- Casos negativos (errores esperados)
- Casos limite
- Validaciones de autenticacion

Para cada caso incluye:
- ID, Descripcion, Precondiciones
- Request completo (metodo, URL, headers, body)
- Resultados esperados (status, campos, validaciones)
```

> **Validacion Humana Requerida**
>
> La IA genera excelentes puntos de partida, pero siempre debes revisar y ajustar los casos. La IA no conoce tus requisitos de negocio especificos ni las restricciones de tu sistema.

2.3 Generacion de Datos Mock

Los datos de prueba realistas son esenciales. Usa IA para generar datasets variados:

Prompt para Datos Mock

```
Genera 20 usuarios de prueba en formato JSON con los siguientes campos:
- id: numero secuencial (1-20)
- nombre: nombres reales de Latinoamerica
- email: formato valido basado en el nombre
- edad: entre 18 y 70 anos
- ciudad: ciudades reales de Mexico, Colombia, Argentina
- activo: booleano aleatorio

Requisitos:
- Incluye caracteres especiales (tildes, n)
- Algunos emails con dominios inusuales
- 2-3 usuarios con datos incompletos para pruebas negativas
- Formato: Array JSON valido
```

Ejemplo de output esperado:

```
[
  {
    "id": 1,
    "nombre": "Maria Fernanda Lopez",
    "email": "mfernanda.lopez@correo.mx",
    "edad": 34,
    "ciudad": "Ciudad de Mexico",
    "activo": true
  },
  {
    "id": 2,
    "nombre": "Jose Antonio Garcia",
    "email": "jose.garcia1985@test.com",
    "edad": 39,
    "ciudad": "Guadalajara",
    "activo": false
  }
]
```

3. Bruno: Guia Completa

3.1 Instalacion y Primeros Pasos

Paso 1: Descargar e Instalar

1. Visita `https://www.usebruno.com`
2. Descarga la version para tu sistema operativo (Windows, macOS, Linux)
3. Ejecuta el instalador
4. Abre Bruno

Paso 2: Crear tu Primer Workspace

```
File → New Workspace
Nombre: "API-Testing-Workshop"
Ubicacion: Elige una carpeta de tu preferencia
```

Paso 3: Crear una Collection

```
Click derecho en el Workspace → New Collection
Nombre: "JSONPlaceholder-Tests"
Descripcion: "Pruebas de API con datos de ejemplo"
```

3.2 Workspaces y Collections

Estructura de Archivos de Bruno

Bruno guarda todo como archivos de texto plano en formato `.bru`. Esto hace que sea Git-friendly:

```
mi-workspace/
├── bruno.json              # Configuracion del workspace
├── collection.bru          # Configuracion de la coleccion
├── environments/
│   ├── development.bru
│   └── production.bru
└── JSONPlaceholder-Tests/
    ├── folder.bru
    ├── users/
    │   ├── get-users.bru
    │   ├── get-user-by-id.bru
    │   ├── create-user.bru
    │   └── update-user.bru
    └── posts/
        ├── get-posts.bru
        └── create-post.bru
```

Ventaja Git-Friendly

Al ser archivos de texto, puedes versionar tus colecciones con Git, hacer code reviews de tus tests, y compartir facilmente con tu equipo sin exportar/importar archivos JSON gigantes.

3.3 Variables y Environments

Tipos de Variables en Bruno

Tabla 5 Tipos de Variables en Bruno

Tipo	Alcance	Uso	Ejemplo
Environment	Toda la coleccion	URLs, tokens, configuracion por ambiente	{{baseUrl}}
Collection	Toda la coleccion	Valores compartidos entre requests	{{apiVersion}}
Request	Request actual	Valores temporales de una sola request	{{userId}}

Creando Environments

En la pestana "Environments", crea dos environments:

Development

```
{
  "baseUrl": "https://jsonplaceholder.typicode.com",
  "apiKey": "dev-key-123",
  "timeout": 5000
}
```

Production

```
{
  "baseUrl": "https://api.production.com",
  "apiKey": "prod-key-456",
  "timeout": 10000
}
```

Usando Variables en Requests

```
// URL con variable
GET {{baseUrl}}/users/1

// Header con variable
Authorization: Bearer {{apiKey}}

// Body con variable
{
  "name": "Test User",
  "email": "test{{$randomInt}}@example.com"
}
```

3.4 Pre-scripts y Test Scripts

Pre-request Scripts

Se ejecutan ANTES de enviar la request. Usalos para:

- Generar datos dinamicos
- Configurar variables temporales
- Obtener tokens de autenticacion

Ejemplo: Generar Email Unico

```
// Pre-request Script
const timestamp = new Date().getTime();
const uniqueEmail = `test${timestamp}@example.com`;
bru.setVar("uniqueEmail", uniqueEmail);

// En el body de la request:
{
  "name": "Test User",
  "email": "{{uniqueEmail}}"
}
```

Test Scripts (Assertions)

Se ejecutan DESPUES de recibir la response. Usalos para validar:

Ejemplo: Validar Response Completa

```
// Test Script

// 1. Validar Status Code
bru.test("Status code is 200", () => {
  expect(res.getStatus()).to.equal(200);
});

// 2. Validar Content-Type
bru.test("Content-Type is JSON", () => {
  expect(res.getHeader("content-type"))
    .to.include("application/json");
});

// 3. Validar estructura del body
bru.test("Response has required fields", () => {
  const body = res.getBody();
  expect(body).to.have.property("id");
  expect(body).to.have.property("name");
  expect(body).to.have.property("email");
});

// 4. Validar tipos de datos
bru.test("Data types are correct", () => {
  const body = res.getBody();
  expect(body.id).to.be.a("number");
  expect(body.name).to.be.a("string");
  expect(body.email).to.include("@");
});

// 5. Validar valores especificos
bru.test("User ID matches request", () => {
  const body = res.getBody();
  expect(body.id).to.equal(1);
});
```

API Flow Testing: Login + Request

Para flujos que requieren autenticacion:

```
// Pre-request Script: Obtener token
const loginRes = await bru.sendRequest({
  url: "{{baseUrl}}/auth/login",
  method: "POST",
  body: {
    username: "testuser",
    password: "testpass"
  }
});

// Guardar token para usar en esta request
bru.setVar("authToken", loginRes.token);

// En Headers:
// Authorization: Bearer {{authToken}}
```

3.5 Data-Driven Testing

Bruno permite ejecutar la misma request con multiples sets de datos usando archivos CSV.

Paso 1: Crear archivo CSV

```
name,email,expectedStatus
"Ana Garcia","ana@test.com",201
"","invalid-email",400
"Luis Martinez","luis@company.com",201
"Maria","maria",400
```

Paso 2: Configurar Iteraciones

1. Abre tu request
2. Ve a la pestana "Runner"
3. Selecciona "Iterations"
4. Carga tu archivo CSV
5. Usa las variables en tu request: `{{name}}` , `{{email}}`

Paso 3: Ejecutar

```
// Body usando variables CSV
{
  "name": "{{name}}",
  "email": "{{email}}"
}

// Test usando expectedStatus
bru.test("Status matches expected", () => {
  expect(res.getStatus()).to.equal(Number("{{expectedStatus}}"));
});
```

Casos de Uso para Data-Driven Testing

- Regresion: Reejecutar todos los casos con cada release
- Smoke tests: Verificar endpoints criticos
- Pruebas de limite: Probar valores frontera
- Pruebas negativas: Verificar manejo de errores

3.6 Exportar y Compartir Collections

Metodo 1: Git (Recomendado)

```
# Inicializar repo en tu workspace
cd mi-workspace
git init
git add .
git commit -m "Initial collection"
git remote add origin https://github.com/user/bruno-collection.git
git push -u origin main
```

Metodo 2: Exportar a Postman

```
Collection → Export → Postman Collection v2.1
// Genera un archivo .json compatible con Postman
```

Metodo 3: Compartir Archivos .bru

Simplemente comprime la carpeta de tu collection y compartela. Los archivos .bru son legibles y autocontenidos.

4. Buenas Practicas y Errores Comunes

Buenas Practicas

Tabla 6 Buenas Practicas de API Testing

Practica	Descripcion	Beneficio
Nombres descriptivos	Usa nombres que describan el proposito del test	Facil mantenimiento y comprension
Variables para todo	No hardcodees URLs, tokens o datos	Portabilidad entre ambientes
Tests atomicos	Cada test debe ser independiente	Facil debug y paralelizacion
Validaciones completas	Valida status, headers y body	Mayor confianza en los resultados
Organizacion por carpetas	Agrupa requests por funcionalidad	Escala mejor con proyectos grandes

Errores Comunes

Error 1: No validar el status code

Incorrecto: Solo verificar que hay response body.
Correcto: Siempre validar `res.getStatus()` primero.

Error 2: Hardcodear datos sensibles

Incorrecto: Poner tokens o passwords directo en el script.
Correcto: Usar variables de environment.

Error 3: Tests dependientes entre si

Incorrecto: Test B asume que Test A creo un recurso.
Correcto: Cada test debe ser autocontenido.

Error 4: Ignorar headers importantes

Incorrecto: Solo validar body.
Correcto: Validar Content-Type, cache headers, etc.

Error 5: No manejar timeouts

Incorrecto: Dejar timeout por defecto.
Correcto: Configurar timeout apropiado por ambiente.

Checklist Final

- Todos los tests pasan?
- Las variables estan correctamente configuradas?
- No hay datos sensibles hardcodeados?
- Los nombres son descriptivos?
- La coleccion esta versionada en Git?